

CODE SPORT



In this month's column, we continue our discussion on the testing of machine learning applications, focusing our discussion on NLP.

In the last two columns, we have been discussing how we can test machine learning applications. In this column, we continue this discussion, with a focus on testing natural language processing (NLP) applications. Voice and text based applications are becoming critical in the deployment of AI in the real world. When we build NLP applications, how can we ensure that they are ready for deployment in the real world? How can we test our NLP applications and make sure that they will not fail in production?

While a number of success stories of the application of NLP have emerged, there have also been a number of newsworthy failures in translating NLP research to real world usage. Microsoft's Twitterbot Tay was released to have realistic conversations with social media users, but ended up being removed just a couple of days after the launch since it picked up profanity and hate speech from the users who were talking with it. There has been news of AI driven algorithms mistakenly driving stock prices up or down due to their inability to understand ambiguous news contexts. For instance, the stock price of Berkshire Hathaway was impacted whenever the actress Anne Hathaway was in the news, because the NLP algorithm was not able to disambiguate between Hathaway as a company entity and Hathaway as a person. While news such as this has made the headlines, there have been many more quietly buried failures of NLP models in real world deployment. What causes these NLP models to fail when they are launched into production?

Our discussion on this topic will be in the context of a supervised learning paradigm. (Similar issues exist in unsupervised and self-supervised learning methods.) Given that most of the current NLP applications are built using the supervised

learning approach, we focus on these first.

Typically, NLP models are built using a training data set. The training data is supposed to comprise data that is representative of the real world inputs, and which has been labelled by domain experts. Given that data collection and annotation takes time, this data is typically not current or recent, but a historical slice of the application's possible inputs in the real world. The NLP model is built on the assumption that this is typically the universe of data that your application needs to deal with. But this representative historical data is typically static, while real world data keeps changing. In addition, training data quality is often taken for granted without extensive validation. In the case of supervised paradigm, test oracles are not easily available and expensive to build.

Let us assume that we are building one of the fairly mature NLP applications, namely, sentiment analysis. Let us also suppose that our task is the binary classification of movie reviews as positive or negative. We have a set of data we have obtained from movie review websites such as IMDb, Rotten Tomatoes etc. The first step is to get this data annotated or labelled for our task. We will then split this data set into training data to train our model and test data, respectively, to evaluate our model. Now, even if you could get access to many more movie reviews online, you don't have a ready-made test oracle to get their correct labels. Hence, we are limited by the size of the labelled data we have obtained for our task on hand.

Today, an NLP model is evaluated by performance metrics such as accuracy, precision, recall, and F1-score on one or more test data sets. However, performance summaries on a few specific data sets are not a good proxy for the model